

TRABAJO PRÁCTICO — DOCUMENTACIÓN TÉCNICA

Pantone Mercedes

Plataforma web de gestión y entrega de fotografía profesional
Arquitectura, autenticación y decisiones de implementación

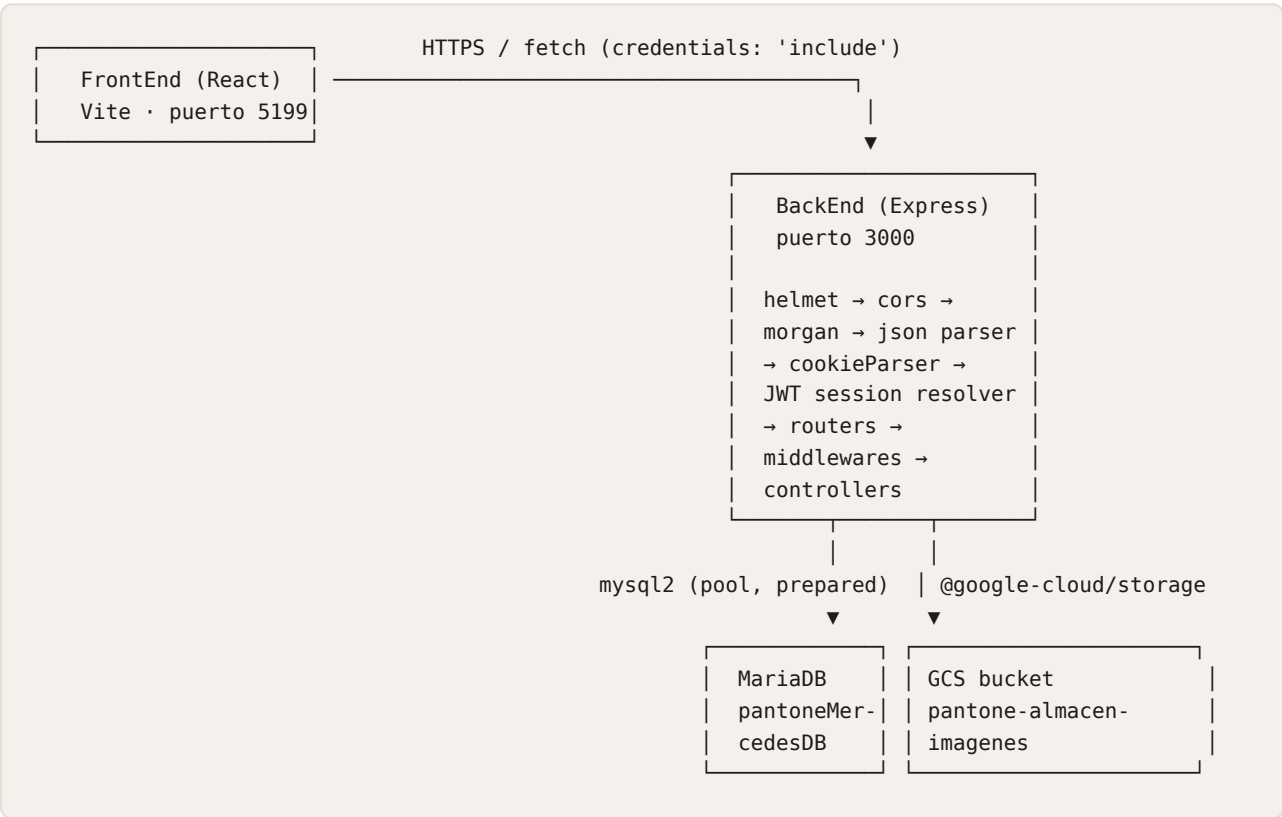
Institución	Universidad del Salvador (USAL)
Stack	React + Vite / Node.js + Express / MariaDB / Google Cloud Storage
Repositorio	PantoneMercedesUSAL
Fecha	Julio 2026

1. Introducción y alcance

Este documento describe la arquitectura técnica de **Pantone Mercedes**, una aplicación web full-stack desarrollada para un estudio de fotografía. Permite a los clientes acceder a sus galerías privadas, seleccionar imágenes, subir canciones para sus videos y hacer seguimiento de solicitudes, mientras que el panel de administración gestiona clientes, galerías, videos y estadísticas.

El objetivo de este documento es exponer las decisiones de diseño técnico tomadas durante el desarrollo — en particular el manejo de autenticación con cookies, el sistema de middlewares y la arquitectura en capas del backend — como evidencia de la autoría e implementación del proyecto.

2. Arquitectura general



El backend sigue una arquitectura en capas clásica de Express: **routes** → **middleware** → **controllers** → **moduls** (capa de acceso a datos) → **database** (pool de conexión MySQL). Cada capa tiene una responsabilidad única, lo que permite reutilizar middlewares (autenticación, logging, subida de archivos) entre distintos módulos del panel de administración.

Carpeta	Responsabilidad
src/routes/	Define endpoints y encadena middlewares antes de llegar al controller (ej. <code>adminRoutes.js</code> , <code>authRoutes.js</code>)
src/middleware/	Lógica transversal: autenticación (<code>auth.js</code>), logging de acciones admin (<code>adminLogger.js</code>), subida de imágenes/video (<code>upload.js</code> , <code>uploadVideo.js</code>)
src/controllers/	Orquestan la lógica de negocio de cada request y arman la respuesta HTTP

src/modules/	Capa de acceso a datos: clases estáticas con queries SQL parametrizadas (<code>Users.js</code> , <code>Galleries.js</code> , etc.)
src/database/	<code>dbConnect.js</code> — pool de conexiones <code>mysql2/promise</code>
src/utils/	Servicios externos, como <code>googleStorageService.js</code> (Google Cloud Storage)

3. Autenticación y manejo de cookies

La sesión no se guarda en el servidor (no hay `express-session`): se implementó un esquema **stateless con JWT de doble token**, almacenado en cookies `httpOnly`. Esto evita que el token sea accesible desde JavaScript en el cliente (mitiga XSS) y evita mantener estado de sesión en memoria/DB.

3.1 Login — emisión de tokens

Backend/src/controllers/authController.js

```
const accessToken = jwt.sign(
  { id: result.id, role: result.role },
  process.env.SECRET_WEB_TOKEN,
  { expiresIn: '1h' }
);
const refreshToken = jwt.sign(
  { id: result.id, role: result.role },
  process.env.REFRESH_TOKEN_SECRET,
  { expiresIn: '30d' }
);
return res
  .cookie('access_token', accessToken, {
    httpOnly: true, sameSite: 'strict', maxAge: 1000 * 60 * 60
  })
  .cookie('refresh_token', refreshToken, {
    httpOnly: true, sameSite: 'strict', maxAge: 1000 * 60 * 60 * 24 * 30
  })
  .status(200).json({ role: result.role });
```

Por qué dos tokens: el `access_token` dura 1 hora y es el que se valida en cada request. El `refresh_token` dura 30 días y sólo se usa para renovar el access token cuando expira, sin obligar al usuario a loguearse de nuevo. Ambas cookies son `httpOnly` (inaccesibles desde JS del navegador) y `sameSite: 'strict'` (no se envían en requests cross-site, mitigando CSRF).

3.2 Resolución de sesión en cada request

En vez de repetir la verificación del JWT en cada controller, hay un middleware global registrado antes de las rutas que decodifica las cookies y arma `req.session`. Si el `access_token` expiró pero el `refresh_token` sigue vigente, se emite un nuevo access token de forma transparente para el usuario.

Backend/app.js

```

app.use((req, res, next) => {
  const token = req.cookies.access_token;
  const refreshToken = req.cookies.refresh_token;
  req.session = { user: null };

  try {
    const data = jwt.verify(token, process.env.SECRET_WEB_TOKEN);
    req.session.user = data;
  } catch (error) {
    if (refreshToken) {
      try {
        const refreshData = jwt.verify(refreshToken, process.env.REFRESH_TOKEN_SECRET);
        const newAccessToken = jwt.sign(
          { id: refreshData.id, role: refreshData.role },
          process.env.SECRET_WEB_TOKEN,
          { expiresIn: '1h' }
        );
        res.cookie('access_token', newAccessToken, {
          httpOnly: true, sameSite: 'strict', maxAge: 1000 * 60 * 60
        });
        req.session.user = { id: refreshData.id, role: refreshData.role };
      } catch (refreshError) {
        // refresh también expirado → sin sesión
      }
    }
  }
  next();
});

```

3.3 Hasheo de contraseñas

Las contraseñas nunca se guardan en texto plano: se hashean con **bcrypt** (10 salt rounds) al registrar, y se comparan con `bcrypt.compare` al hacer login.

```

const hashPassword = await bcrypt.hash(password, 10);
...
const validPassword = await bcrypt.compare(password, result.password);

```

3.4 Logout

El logout simplemente limpia ambas cookies del navegador — al no haber estado de sesión en el servidor, no hace falta invalidar nada en base de datos.

```

res.clearCookie('access_token').clearCookie('refresh_token')
  .status(200).json({ message: 'Logout Exitoso' });

```

4. Middlewares personalizados

4.1 Control de acceso por rol — `auth.js`

Dos guardas reutilizables que se enganchan a nivel de router. Leen `req.session.user` (ya resuelto por el middleware global de JWT) y cortan la cadena con `401` si no corresponde.

`BackEnd/src/middleware/auth.js`

```
const requireAuth = (req, res, next) => {
  if (!req.session.user) return res.status(401).json({ message: 'Acceso denegado' });
  next();
};

const requireAdmin = (req, res, next) => {
  if (!req.session.user || req.session.user.role !== 'admin')
    return res.status(401).json({ message: 'Acceso no autorizado' });
  next();
};
```

Se aplica a nivel de router completo, por ejemplo todas las rutas de `/admin` quedan protegidas con una sola línea:

```
// BackEnd/src/routes/adminRoutes.js
router.use(requireAdmin);
```

4.2 Middleware de auditoría — `adminLogger.js`

Middleware de **orden superior** (una función que devuelve un middleware) que registra automáticamente cada acción administrativa en la tabla `admin_activity_logs`: quién, qué acción, sobre qué recurso, IP y user-agent. Intercepta `res.json` para loggear **después** de que la respuesta ya fue enviada, sin bloquear ni afectar la respuesta al cliente.

```
const withAdminLog = (action_type, getResourceDataFn = null) => {
  return async (req, res, next) => {
    const originalJson = res.json;
    res.json = async function(data) {
      originalJson.call(this, data);
      if (res.statusCode >= 200 && res.statusCode < 300) {
        const resourceData = getResourceDataFn ? getResourceDataFn(req, data) : {};
        await logAdminAction(req, action_type, resourceData.resource_type, ...);
      }
    };
    next();
  };
};
```

Uso típico en una ruta, pasando una función que extrae datos de la request/respuesta para el log:

```
router.post('/createClient',
  withAdminLog('CLIENT_CREATE', (req, data) => ({
    resource_type: 'CLIENT',
    resource_id: data.data?.id,
    resource_name: `${req.body.first_name} ${req.body.last_name}`
  })),
  adminController.createClient
);
```

4.3 Subida de archivos — `upload.js` / `uploadVideo.js`

Se usa **Multer** en modo `memoryStorage` (el archivo nunca toca el disco del servidor, se sube directo al buffer y de ahí a Google Cloud Storage). Incluye validación de tipo MIME/extensión y límites de tamaño/cantidad, con un segundo middleware para traducir errores de Multer a respuestas JSON legibles en vez del stack trace crudo.

```
const uploadGaleria = multer({
  storage: multer.memoryStorage(),
  fileFilter, // sólo jpeg/jpg/png/gif/webp
  limits: { fileSize: 50 * 1024 * 1024, files: 50 }
}).array('images', 50);

const handleUploadErrors = (error, req, res, next) => {
  if (error instanceof multer.MulterError) {
    if (error.code === 'LIMIT_FILE_SIZE')
      return res.status(400).json({ success:false, message:'Imagen demasiado grande (máx 50MB)' });
    ...
  }
  next();
};
```

5. Seguridad implementada

Capa	Mecanismo	Detalle
Headers HTTP	<code>helmet()</code>	Cabeceras de seguridad por defecto (CSP básica, no-sniff, etc.) — <code>app.js</code>
CORS	Whitelist explícita	Sólo se acepta <code>origin</code> desde los puertos de desarrollo del frontend, con <code>credentials: true</code> para permitir el envío de cookies cross-port
Contraseñas	<code>bcrypt</code> (10 rounds)	Nunca se persiste texto plano
Tokens	JWT firmado + <code>httpOnly</code> / <code>sameSite: strict</code>	Inaccesible desde JS, resistente a CSRF básico
SQL	<code>mysql2</code> prepared statements	Todas las queries en <code>src/moduls/</code> usan placeholders <code>?</code> , nunca concatenación de strings — previene inyección SQL
Errores	Error handler global	Middleware final en <code>app.js</code> evita filtrar stack traces al cliente

BackEnd/app.js — CORS

```
app.use(cors({
  origin: (origin, callback) => {
    const allowed = ['http://localhost:5173', 'http://localhost:5174', 'http://localhost:5175', 'http://localhost:5176'];
    if (!origin || allowed.includes(origin)) callback(null, true);
    else callback(new Error('Not allowed by CORS'));
  },
  credentials: true
}));
```

Ejemplo de query parametrizada — BackEnd/src/moduls/Users.js

```
static async equalPassword(email){
  const [result] = await pool.execute('SELECT * FROM users WHERE email = ? ', [email]);
  return result[0];
}
```

6. Persistencia de datos

El acceso a MariaDB se hace con un **pool de conexiones** de `mysql2/promise`, instanciado una sola vez y reutilizado en toda la app. Cada entidad del dominio (usuarios, galerías, videos, comentarios, notificaciones, etc.) tiene su propia clase con métodos estáticos en `src/modules/`, que encapsulan las queries SQL.

```
// Backend/src/database/dbConnect.js
const pool = mysql.createPool({
  host: "127.0.0.1", user: "root", password: process.env.DB_PASS,
  database: "pantoneMercedesDB"
});
module.exports = pool;
```

7. Almacenamiento de archivos — Google Cloud Storage

Las imágenes de galerías y videos no se guardan en el servidor: se suben directo a un bucket de GCS (`pantone-almacen-imagenes`) desde `googleStorageService.js`, usando el buffer en memoria que entrega Multer. El controller sólo persiste en MariaDB la URL pública resultante.

```
const BUCKET_NAME = process.env.BUCKETNAME || 'pantone-almacen-imagenes';
const bucket = storage.bucket(BUCKET_NAME);
...
const publicUrl = `https://storage.googleapis.com/${BUCKET_NAME}/${fileName}`;
```

8. Integración Frontend ↔ Backend

El frontend (React + Vite) consume la API con `fetch` nativo, sin librerías externas como axios. La clave para que las cookies `httpOnly` viajen en cada request es `credentials: 'include'`, presente en todas las llamadas del panel de admin y del dashboard de cliente:

```
// FrontEnd/src/assets/Components/Admin/AdminDashboard.jsx
const res = await fetch(`${API_URL}/admin/dashboard-stats`, {
  credentials: 'include'
});
```

Como el backend resuelve el rol del usuario a partir del JWT en la cookie (no de un header manual), el frontend no necesita manejar tokens en JavaScript ni guardarlos en `localStorage` — reduce la superficie de ataque XSS.

9. Resumen de decisiones de diseño

- **JWT en cookies `httpOnly` en vez de `localStorage`:** evita que un ataque XSS pueda robar el token directamente desde JS.
- **Doble token (`access` + `refresh`):** balancea seguridad (sesiones cortas de 1h) con experiencia de usuario (no relogueo cada hora).
- **Middleware de resolución de sesión centralizado en `app.js`:** evita repetir lógica de verificación de JWT en cada controller.
- **Guards `requireAuth` / `requireAdmin` aplicados a nivel de router:** protege secciones completas de la API con una sola línea, en vez de por endpoint.

- **Middleware de auditoría desacoplado (`withAdminLog`)**: permite loggear acciones administrativas sin ensuciar la lógica de cada controller, reutilizando el mismo wrapper en decenas de rutas.
- **Multer en memoria + subida directa a GCS**: el servidor de la app nunca almacena archivos en disco, lo que simplifica el despliegue y evita problemas de espacio/persistencia.
- **Prepared statements en toda la capa de datos**: previene inyección SQL de forma sistemática.

Documento generado a partir del código fuente del repositorio `PantoneMercedesUSAL` (BackEnd/FrontEnd), con referencias directas a archivo y fragmento de código para cada mecanismo descripto.